



МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний університет "Києво-Могилянська академія"

Факультет Інформатики

Програма "Створення схем української вишивки засобами Java"

Студентки 1 курсу групи
спеціальності F2. Інженерія
програмного забезпечення
Солтис Тетяни

Викладачка:
Печкурова Олена Миколаївна

м. Київ – 2026

Зміст

Зміст	1
ПОСТАНОВКА ЗАДАЧІ	3
Ключові архітектурні модулі та розширені вимоги	3
Опис класів та методів	4
Технічний опис можливостей та бізнес-логіки	4
Ключові методи та алгоритми системи	6
Інструкція для користувача	6
Проблеми, що виникали і їх вирішення	11
Висновок.....	13
Програмний код	14

ПОСТАНОВКА ЗАДАЧІ

Тема: Інтерактивний графічний редактор орнаментів української вишивки.

Мета роботи:

Розробити програму для створення схеми української вишивки. Користувач повинен мати змогу інтерактивно обирати кольори з палітри та малювати на сітці клітинок, формуючи власний візерунок. Дані вишивки зберігаються у внутрішній структурі (масив або список).

Формат виконання та предметна область:

Робота ведеться індивідуально з обов'язковим контролем версій у системі Git із регулярними комітами, з використанням різних гілок та Pull Requests.

Застосунок оперує двовимірним дискретним полотном (матрицею клітинок), де кожен елемент (хрестик) є самостійною бізнес-сутністю, що має просторові координати та параметри кольору нитки. Система повинна забезпечувати безперервний графічний цикл відтворення (rendering loop), захист від некоректного введення даних та повне покриття функцій гарячими клавішами для досягнення максимального показника ергономіки системи (UI/UX).

Ключові архітектурні модулі та розширені вимоги

Проект спроектовано за принципом низької зв'язності (Low Coupling) та високої згуртованості (High Cohesion), що розподіляє систему на такі функціональні пласти:

Доменна модель (Domain Model): Опис просторових сутностей. Використано компактні вкладені структури (Stitch) для оптимізації споживання оперативної пам'яті за великої кількості нанесених елементів.

Persistence Layer (Шар довготривалого зберігання): Модуль прямого та зворотного перетворення об'єктного графа у растровий формат PNG. Реалізує алгоритм сканування матриці пікселів та розпізнавання меж полотна для відновлення стану об'єктів у пам'яті JVM після перезапуску системи.

Модуль обробки подій та гарячих клавіш: Глибока інтеграція системних мап InputMap та ActionMap третього рівня Swing для глобального перехоплення фокусу клавіатури, що виключає конфлікти периферії.

Математичний модуль симетрії: Обчислення просторових дзеркальних інваріантів відносно динамічних осей матриці екрана в реальному часі.

Опис класів та методів

Система складається з 5 окремих класів:

1. **Stitch (Модель даних):** Базова інформаційна сутність, яка описує один вишитий хрестик. Містить лише координати розташування у сітці (gridX, gridY) та колір нитки (Color color).
2. **EmbroideryCanvas:** Клас успадковує JPanel і відповідає виключно за рендеринг графіки та відстеження дій миші.
 - a. *Метод paintComponent():* Здійснює подвійну буферизацію, відмальовує біле полотно, сітку, масив хрестиків та сервісні рамки (синю рамку виділення, клавіатурний курсор).
 - b. *Метод floodFill():* Реалізує рекурсивний алгоритм заливки.
 - c. *Метод handleMouseClicked():* Транслює піксельні координати кліку в індекси матриці та ініціює відповідну подію (малювання, стирання, вибір кольору).
3. **FileManager (Модуль вводу/виводу):** Ізолює в собі логіку роботи з файловою системою.
 - a. *Метод saveToPNG():* Формує BufferedImage та записує графічний контекст полотна у файл за допомогою ImageIO.
 - b. *Метод openFromPNG():* Реалізує алгоритм попиксельного сканування, розпізнає межі тканини та конвертує кольорові пікселі назад в об'єкти Stitch.
4. **ToolManager:** Відповідає за виклик модальних вікон інтерфейсу користувача.
 - a. *Метод chooseColor():* Ініціалізує стандартний компонент JColorChooser.
 - b. *Метод duplicateFragment():* Формує кастомну панель з JSpinner для задання ширини та висоти фрагмента перед його клонуванням.
5. **Main:** Головний інфраструктурний клас. Запускає потік SwingUtilities.invokeLater(), збирає JFrame, ініціалізує всі вищезгадані класи та зв'язує їх між собою. Саме тут налаштовується верхня панель JToolBar і реєструється глобальна мапа гарячих клавіш, що передають команди до EmbroideryCanvas.

Технічний опис можливостей та бізнес-логіки

В межах розробленого графічного редактора було реалізовано наступний комплекс можливостей:

2.1 Інтелектуальний UX-інструмент дублювання орнаменту

На відміну від статичного копіювання, реалізовано інтерактивний механізм вибору базового фрагмента:

- Програма відкриває модальне діалогове вікно з лічильниками чисел JSprinner, де користувач чітко задає межі області (наприклад, 5 x 7 або 10 x 10 клітинок).
- Після затвердження розмірів система переходить у **Режим вибору області** (State Mode). На полотні виводиться напівпрозорий синій обмежувальний прямокутник заданого розміру.
- Рух миші динамічно перераховує координати рамки на дискретній сітці полотна. Клік ЛКМ ініціює миттєве захоплення елементів, що потрапили всередину рамки, очищення полотна та циклічне заомощення («встик») всієї матриці полотна (41 x 29) з автоматичним накладанням поточної симетрії.

2.2 Багаторівневий буфер скасування дій (Undo Engine)

Для вирішення проблеми випадкових кліків користувача спроектовано кастомний менеджер історії кроків:

- Оскільки при активній симетрії один клік миші або натискання пробілу генерує від 1 до 4 хрестиків одночасно, звичайне видалення останнього елемента з масиву призвело б до руйнування симетричного малюнка.
- Створено динамічний числовий стек undoHistory (масив ArrayList<Integer>). При кожній транзакції малювання система фіксує розмір масиву хрестиків до і після події, записуючи чисту різницю (кількість створених хрестиків) в історію.
- При виконанні команди «**Назад**» система дістає останнє число зі стеку історії та видаляє саме таку кількість елементів з кінця основного об'єктного списку, забезпечуючи математично точне скасування всього кроку малювання.

2.3 Реверс-інжиніринг та парсинг растрових графічних схем

Модуль завантаження PNG реалізує складний алгоритм розпізнавання:

- При відкритті збереженого растрового зображення система не просто виводить картинку як статичний фон, а попіксельно сканує її за допомогою матричного аналізу image.getRGB(x, y).

- Алгоритм автоматично вираховує точні межі робочого білого полотна всередині картини для визначення динамічних відступів `offsetX` та `offsetY`.
- Після знаходження точки відліку система виконує крок, що дорівнює константі `CELL_SIZE`, зчитує колір центрального пікселя кожної клітинки, відсікає білий колір фону за колірним порогом (`Color.getRed() < 245`) і автоматично створює об'єкти `Stitch` з оригінальними кольорами, повністю відновлюючи векторну схему для подальшого редагування.

2.4 Асинхронна генерація стартової анімації

Для візуального ефекту «живого вишивання» при старті програми використовується асинхронний таймер `javax.swing.Timer` із кроком подій у 30 мс. Масив базового орнаменту піддається випадковому перемішуванню за допомогою `Collections.shuffle(allStitches)`. Потік таймера покроково переносить елементи у список відображення, створюючи ефект плавного та хаотичного виникнення хрестиків на тканині.

Ключові методи та алгоритми системи

- `duplicateFragment()`: Активує інтерактивний режим вибору області, ініціалізує лічильники `JSpinner` та переводить систему у стан відстеження миші `isSelectingArea = true`.
- `handleMouseClicked(MouseEvent e)`: Центральний диспетчер подій миші. Приймає координати кліку, перераховує їх у просторові індекси матриці полотна з урахуванням системних відступів. Містить розгалужену логіку: у стані `isSelectingArea` виконує захоплення об'єктів під синьою рамкою, а у звичайному стані реалізує двокнопкове малювання (ЛКМ — нанесення, ПКМ — гумка).
- `drawStitchAtKeyboardCursor()`: Метод-імітатор транзакцій малювання. Зчитує положення клавіатурного курсора `keyboardGridX/Y` та будує навколо нього дзеркальні координати відповідно до поточного стану `symmetryMode`.
- `undo()`: Виконує транзакційне скасування змін екрана на основі зчитування кількості елементів з буфера історії `undoHistory`.
- `moveKeyboardCursor(int dx, int dy)`: Змінює координати клавіатурного курсора, примусово обмежуючи його межами сітки за допомогою умовних операторів для запобігання помилки `IndexOutOfBoundsException`.

Інструкція для користувача

Як запустити проєкт

Варіант 1: Через середовище розробки IntelliJ IDEA

1. Клонуйте або завантажте репозиторій із кодом.
2. Відкрийте папку проєкту в IntelliJ IDEA.
3. Переконайтесь, що JDK підключено (версія 17 або вище).
4. Знайдіть файл Main.java, натисніть на зелений трикутник поруч із методом main або натисніть Shift + F10.

Варіант 2: Через термінал / командний рядок

- Перейдіть у папку з файлами .java
- Виконайте команди:

`javac Main.java EmbroideryCanvas.java`

`java Main`

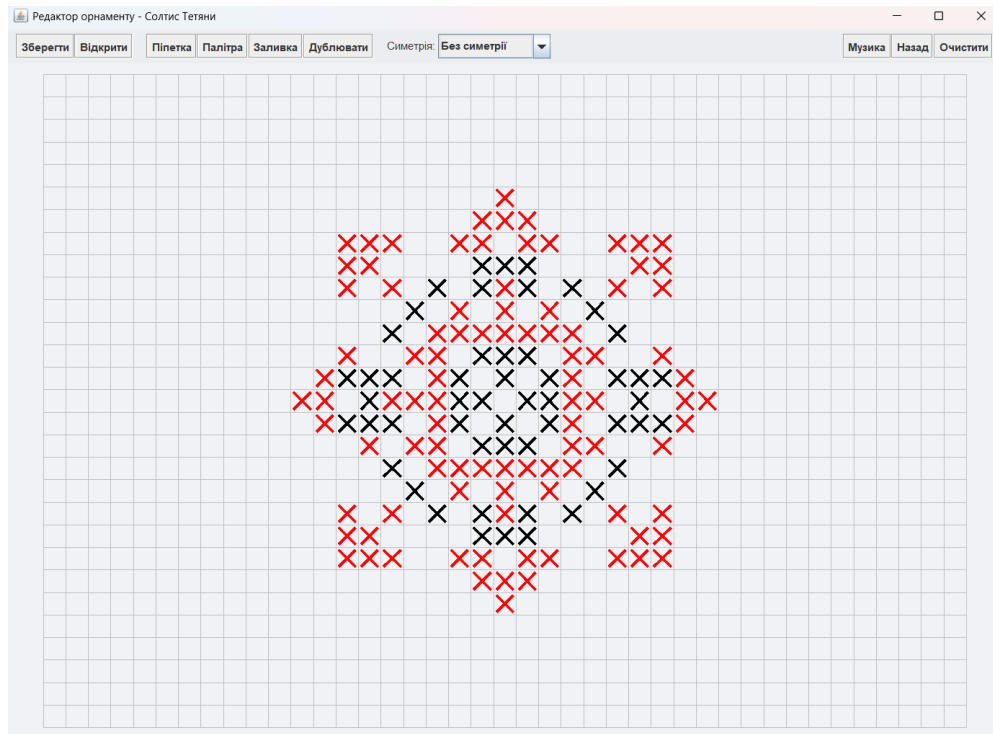
Повна розкладка гарячих клавіш

Для зручності у додатку вимкнено стандартне перехоплення фокусу кнопками панелі інструментів, що дозволяє керувати процесом малювання прямо з клавіатури.

- ↑, ↓, ←, → – навігація по сітці
- Space – нанесення хрестика
- Ctrl + Z – скасувати дію (Undo)
- Ctrl + S – швидке збереження (експорт у PNG)
- P – палітра кольорів (JColorChooser)
- M – керування музикою
- C – очищення полотна (з підтвердженням)
- F – заливка поля
- E – інструмент “Піпетка”
- ESC – скасування вибору

Всі функції та можливості програми

1. Стартова анімація вишивання

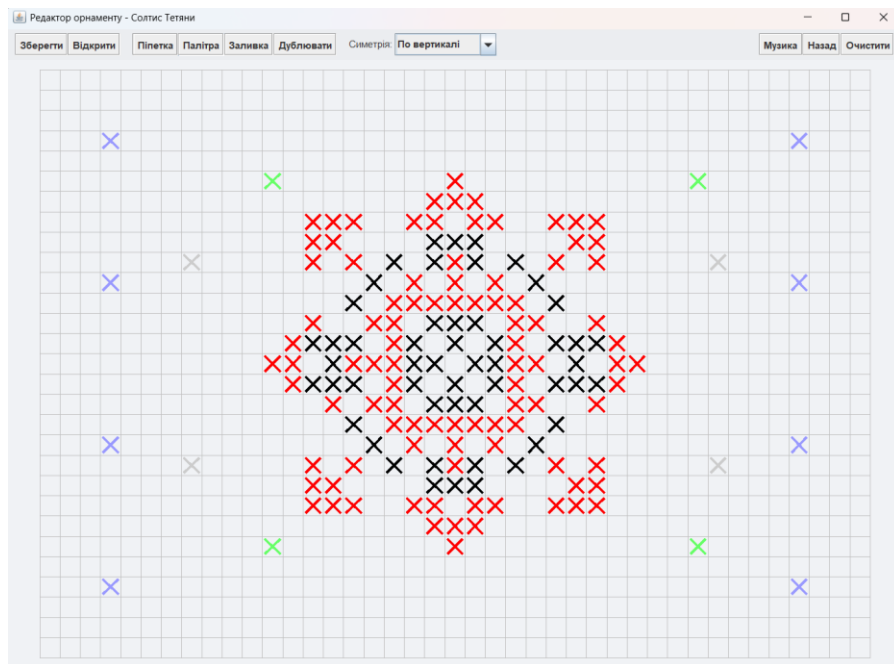


При кожному запуску програми користувач бачить ефектний запуск системи: традиційний український візерунок вишивається самостійно у реальному часі. Використовується `javax.swing.Timer` та випадкове перемішування елементів, що імітує живий процес роботи майстра.

2. Малювання мишею

- Ліва кнопка миші (ЛКМ): малює хрестик обраного кольору.
- Права кнопка миші (ПКМ): працює як гумка, стираючи хрестики.

3. Режими симетрії

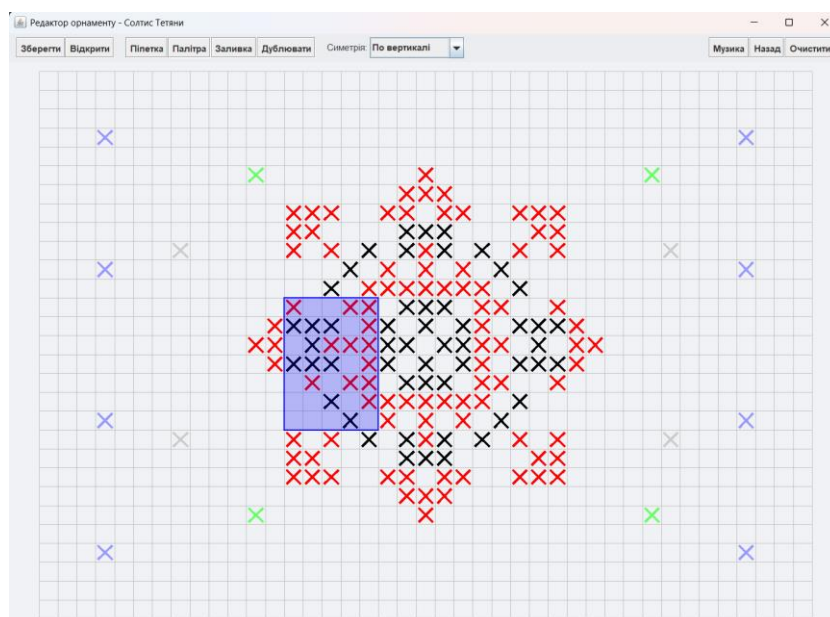


Програма підтримує 4 режими:

- Без симетрії
- По горизонталі
- По вертикалі
- Чотиристороння симетрія

4. Інструмент «Дублювання орнаменту»

Дозволяє автоматично масштабувати та зациклювати візерунки на всю ширину та висоту полотна. Користувач задає розмір рамки, після чого програма розмножує вибраний фрагмент по всій сітці.





5. Багаторівнева історія дій (Undo)

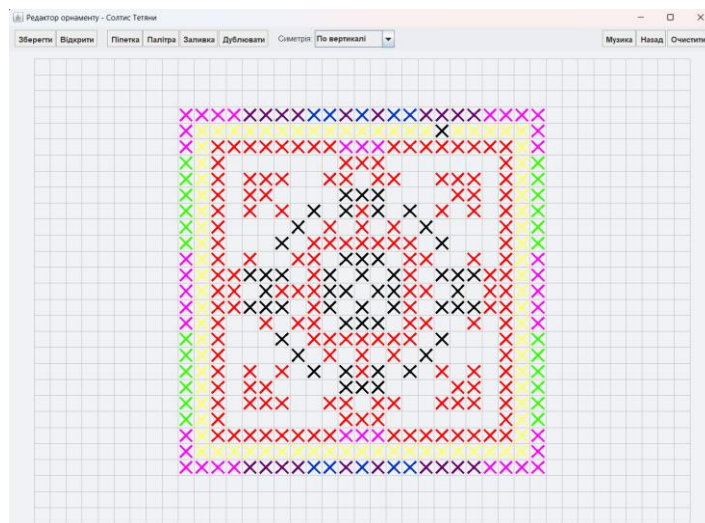
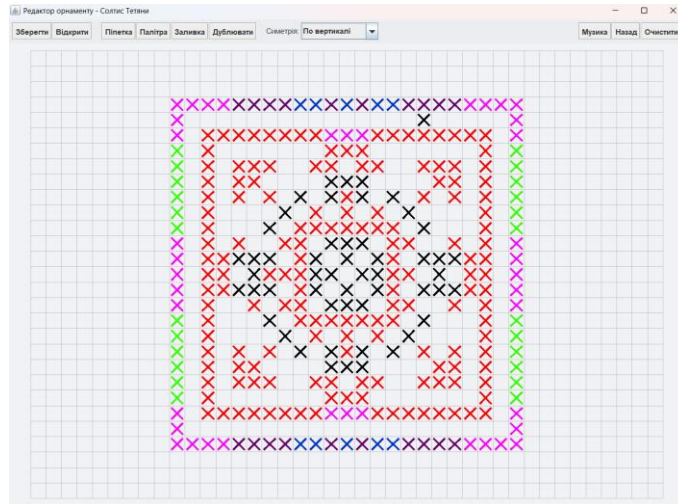
Програма підраховує кількість хрестиків, створених за один клік чи натискання пробілу, і зберігає їх у списку undoHistory. При натисканні Ctrl + Z видаляється саме остання група хрестиків.

6. Імпорт та експорт у PNG

- Експорт: збереження полотна у файл image.png.
- Імпорт: алгоритм реверс-інжинірингу відновлює хрестики з PNG, зчитуючи кольори клітинок і координати.

7. Автоматизований інструмент «Заливка»

- Інструмент призначений для швидкого зафарбовування великих суміжних областей полотна (активація кнопкою «Заливка» або клавішею F).
- Працює на основі рекурсивного алгоритму Flood Fill.
- При кліку ЛКМ на клітинку система аналізує її початковий стан і каскадно перефарбовує всі сусідні клітинки (вгору, вниз, ліворуч, праворуч) ідентичного кольору в новий обраний колір нитки.
- Алгоритм містить захист від нескінченної рекурсії, що унеможливорює зависання програми та виникнення помилки StackOverflowError.



8. Інструмент «Піпетка»

- Інструмент дозволяє миттєво зчитувати кольори безпосередньо з робочої сітки додатка (активація кнопкою «Піпетка» або клавішею E).
- Користувачу достатньо активувати режим та клікнути ЛКМ на будь-який вже вишитий хрестик на полотні — програма зчитає його колір та автоматично встановить його як поточний активний колір нитки для подальшого малювання.
- Це виключає необхідність постійно відкривати модальне вікно «Палітра» під час роботи зі складними багатоколірними орнаментами.

9. Фоновий аудіо супровід

Програма інтегрує систему Java Audio. При натисканні кнопки «Музика» або клавіші M запускається мелодія leleka.wav, яка автоматично зациклюється, створюючи атмосферний фон для роботи.

Проблеми, що виникали і їх вирішення

Блокування гарячих клавіш стрілочок (UP/DOWN/LEFT/RIGHT)

- **Проблема:** При перших спробах реалізації руху клавіатурного курсора за допомогою стрілочок, система Swing повністю ігнорувала натискання клавіш. Замість руху курсора по сітці відбувалося циклічне перемикання та фокусування між кнопками «Зберегти», «Палітра», «Музика» на верхній панелі JToolBar.
- **Вирішення:** Було виявлено, що Swing має жорстку внутрішню специфікацію навігації по компонентах інтерфейсу. Проблема вирішено шляхом примусової деактивації фокусування для всіх без винятку компонентів верхньої панелі та самої JToolBar за допомогою виклику методу `.setFocusable(false)` для кожного об'єкта. Це дозволило передати 100% фокусу клавіатури на кореневу панель вікна `frame.getRootPane()`.

Ефект «нульової рамки» при виборі області

- **Проблема:** Після закриття вікна параметрів дублювання замість великої обмежувальної рамки на екрані відображалася лише мікроскопічна синя крапка розміром в 1 піксель.
- **Вирішення:** Аналіз коду виявив класичну помилку затінення полів класу локальними змінними (`variable shadowing`). Значення лічильників зчитувалися у новостворені локальні змінні `int fragW` та `int fragH` всередині методу, тоді як метод рендерингу `paintComponent` використовував глобальні поля класу `selectionWidth` та `selectionHeight` (які залишалися рівними 0). Помилку усунуто шляхом прямого присвоєння через оператор `=`.

Жорстка прив'язка до палітри при реверс-інжинірингу PNG

- **Проблема:** На початкових етапах розробки модуля читання схем (`openFromPNG`) алгоритм парсингу пікселів мав жорстку прив'язку лише до двох системних кольорів: червоного та чорного. Коли користувач намагався завантажити картинку орнаменту, що містив інші кольори (наприклад, зелений чи синій, вибрані через розширену палітру), програма їх просто ігнорувала, залишаючи полотно порожнім або відтворювала лише те, що було в чорно-червоних тонах.
- **Вирішення:** Було прийнято рішення повністю відв'язати алгоритм розпізнавання від конкретних значень кольорів. Логіку переписали методом виключення: оскільки фоном полотна завжди є білий колір, система аналізує колір центрального пікселя кожної клітинки за RGB-порогом фону (`pixelColor.getRed() < 245 ...`). Якщо колір пікселя відмінний від білого, програма автоматично зчитує його поточний

об'єктивний колір і створює хрестик саме цього кольору. Це дозволило системі коректно імпортувати схеми будь-якої колірної складності.

Порушення принципів інкапсуляції та конфлікти типізації при рефакторингу

- **Опис проблеми:** На перших етапах проектування архітектури програми деякі важливі змінні станів інструментів полотна (такі як режими заливки `isBucketFillMode`, піпетки `isColorPickerMode` та рамки виділення `isSelectingArea`) були оголошені з модифікаторами доступу `public` для швидкого зв'язку між класами `Main` та `EmbroideryCanvas`. Це призвело до архітектурного «зачеплення» (`tight coupling`). Коли код почав розростатися, пряма модифікація цих полів із різних обробників подій (кнопок, гарячих клавіш) почала викликати збої в логіці та численні конфлікти компіляції під час спроб оновити чи змінити поведінку інструментів.
- **Вирішення:** Було проведено повний рефакторинг та інкапсуляцію станів. Усі критичні змінні переведено у статус `private`. Для керування ними в класі `EmbroideryCanvas` створено безпечні відкриті методи-аксесори (наприклад, `activateColorPickerMode()`, `cancelActiveModes()`). Усі зовнішні виклики в класі `Main` було переписано на використання цих методів, що дозволило повністю ізолювати внутрішню логіку полотна від зовнішнього інтерфейсу та уникнути конфліктів доступу. Також монолітний клас полотна було розділено на 5 вузькоспеціалізованих класів, що дозволило повністю позбутися архітектури 'Божественного об'єкта' (`God Object`).

Висновок

У ході виконання курсового проекту було успішно розроблено високотехнологічний інтерактивний графічний редактор схем української вишивки мовою Java із використанням бібліотеки `Swing`. Створений застосунок повністю виконує поставлені завдання, забезпечуючи точний двовимірний рендеринг елементів орнаменту на дискретній сітці.

Окрім базового функціоналу, було значно розширено головні технічні вимоги та інтегровано низку додаткових модулів, які вивели проєкт на рівень повноцінної графічної системи. Зокрема, реалізовано:

- **Ергономічне керування (UI/UX):** впроваджено систему повного дублювання дій клавіатурою (рух графічного курсора стрілочками, нанесення хрестиків через Space та відкриття палітри клавішею P), що дозволяє проєктувати візерунки без обов'язкового використання миші. Для запобігання системних конфліктів Swing було успішно оптимізовано перехоплення фокусу компонентів.
- **Інтерактивні підказки (Tooltip Texts):** кожна кнопка панелі інструментів обладнана динамічним текстовим супроводом, який інформує користувача про призначення елемента та відповідні комбінації гарячих клавіш для швидкого доступу.
- **Мультимедійний супровід:** реалізовано асинхронне зациклене відтворення фонові аудіодоріжки за допомогою Java Sound API, що створює автентичну атмосферу під час творчого процесу.
- **Інтелектуальні алгоритми збереження та реверс-інжинірингу растрових зображень PNG:** програма здатна самостійно аналізувати колірні матриці та відновлювати векторні об'єкти вишивки з файлів будь-якої колірної складності, відійшовши від жорсткої прив'язки до палітри.

Завдяки впровадженим рішенням та двом модальностям взаємодії (миша + клавіатура) розроблена програма стала максимально орієнтованою на користувача, ергономічною, легкою в користуванні та готовою до практичного застосування у сфері цифрового дизайну українського текстилю. Об'єктно-орієнтований підхід та модульна архітектура забезпечують високу стабільність коду та відкривають широкі можливості для його подальшого масштабування.

Програмний код

<https://github.com/tetiana12s/ua-themes.git>